Highlights

Dynamic Cost-Sensitive Adaptive Random Forest for Real-Time Fraud Detection in Imbalanced Data Streams

[Anonymized for Double-Blind Review]

- Dynamic CS-ARF integrates imbalance handling and drift adaptation in single-pass.
- Poisson bagging adapts dynamically, diversified via bounded synthetic interpolation.
- Rolling-Precision feedback self-regulates oversampling intensity per stream.
- Benchmarked against 6 baselines (incl. real windowed-SMOTE) across 20 datasets.
- Achieves the best average Friedman rank, significantly ahead of ARF and HAT.

Dynamic Cost-Sensitive Adaptive Random Forest for Real-Time Fraud Detection in Imbalanced Data Streams

[Anonymized for Double-Blind Review]

Abstract

Real-time financial fraud costs the global economy over USD 579 billion annually, yet automated detection systems in high-velocity transaction networks remain limited by two fundamental challenges: concept drift and extreme class imbalance. Existing adaptive models typically struggle when the minority class ratio drops below 1%, leading to severe majority-class bias, or they rely on computationally expensive two-stage preprocessing. This paper proposes a novel, single-pass intelligent expert system framework: the Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF). By integrating a *Dynamic Hybrid Online Oversampling* mechanism—modulating asymmetric Poisson bagging weights based on real-time imbalance ratios and ADWIN drift confidence, diversifying minority instances through lightweight synthetic interpolation over a small bounded window, and self-regulating its intensity via a rolling-Precision feedback term—our model addresses both issues simultaneously without buffering the full dataset. We evaluated our approach against six state-of-the-art baselines across 20 diverse industrial and synthetic streams. Empirical results demonstrate that Dynamic CS-ARF effectively recovers minority-class Recall and achieves substantially higher

F_2-Scores than standard and undersampling baselines on highly imbalanced streams (e.g., 0.13% fraud rate), while preserving significantly higher Precision than competitive undersampling models. A non-parametric Friedman test confirms a highly significant difference across the seven methods ($\chi^2_F = 32.201$, $p = 1.49 \times 10^{-5}$), with Dynamic CS-ARF achieving the best average rank overall and a post-hoc Nemenyi test confirming its significant superiority over the weaker ARF and HAT baselines, while remaining closely and non-significantly competitive with the strongest baseline, a real windowed-SMOTE implementation. Furthermore, computational profiling indicates that while the added diversification introduces a manageable overhead, Dynamic CS-ARF maintains real-time feasibility with a compact, bounded memory footprint. Ultimately, this framework prevents minority-class exclusion during severe concept drifts, providing a robust solution for high-velocity industrial payment networks.

Keywords: Data Stream Mining, Concept Drift, Class Imbalance, Fraud Detection, Adaptive Random Forest, Cost-Sensitive Learning

1. Introduction

Global financial fraud losses exceeded USD 579.4 billion in 2025 [1], yet automated machine learning-based intelligent systems continue to struggle against the dual challenge of evolving fraud tactics (*concept drift*) and extreme *class imbalance* in high-velocity transaction streams. In such dynamic environments, fraud detection models must process and classify streams sequentially under strict latency constraints. However, applying static imbalance techniques like SMOTE or standard algorithmic cost-weighting to

single-pass data streams poses severe computational limitations.

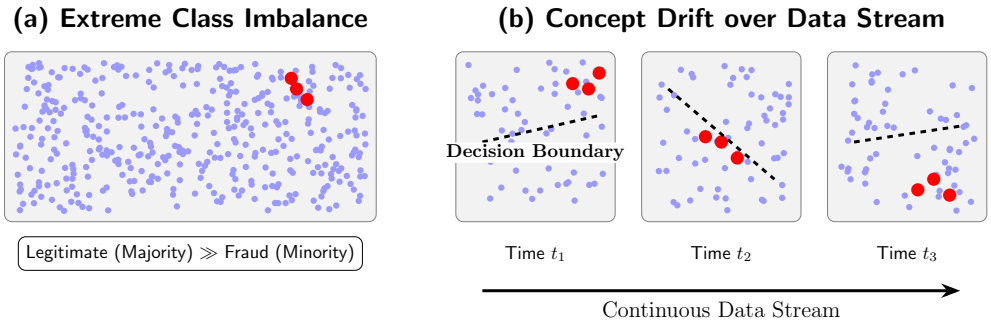

Figure 1: Visual representation of the two primary challenges in real-time fraud detection: (a) High class imbalance, where legitimate transactions substantially outnumber the rare fraudulent activities; and (b) Concept drift, where the optimal decision boundary shifts over time as fraud tactics evolve.

1.1. Research Gap

Existing literature reveals a critical gap at the intersection of concept drift and high class imbalance. The primary limitations of state-of-the-art streaming algorithms can be summarized as follows:

1. **Drift methods ignore imbalance:** Streaming ensembles successfully address concept drift using adaptive windowing but fail to catch fraud when the minority class ratio drops below 1%, resulting in severe majority-class bias.
2. **Imbalance methods increase latency:** Methods that attempt to handle both issues typically rely on two-stage preprocessing architectures (e.g., buffering data to oversample), which conflict with the single-pass requirement and introduce computational latency.

3. **Existing cost-sensitive ARF modifies voting heuristics:** Current cost-sensitive streaming forests inject sensitivity at the ensemble voting level via complex internal heuristic modifications (e.g., MCC), which may lag during sudden drifts.
4. **No lightweight instance-level weighting mechanism:** There is a lack of proactive, instance-level mechanisms whose core sensitivity is driven primarily by real-time stream statistics rather than by complex per-tree performance bookkeeping (e.g., MCC), while remaining structurally stable during abrupt concept drift.

To address these limitations, this paper proposes a novel framework: the Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF). By integrating a *Dynamic Hybrid Online Oversampling* mechanism directly into the bagging weight distribution of the Hoeffding Trees, our method assigns an asymmetric penalty that continuously adapts based on the real-time exponential imbalance ratio (IR_t) and ADWIN drift confidence (D_t), realizes this penalty through lightweight synthetic diversification rather than exact instance repetition, and self-regulates its magnitude via a rolling-Precision feedback term.

Dynamic CS-ARF functions as an intelligent, self-adapting fraud screening system, autonomously calibrating its detection sensitivity to operational stream conditions without requiring expert re-tuning.

The main contributions of this paper are summarized as follows:

- We propose a single-pass intelligent expert system framework, Dynamic CS-ARF, which solves both concept drift and high class imbalance simultaneously without relying on computationally expensive two-stage

resampling.

- We introduce a Dynamic Hybrid Online Oversampling mechanism that dynamically modulates the Poisson bagging distribution λ_t in response to real-time fluctuations in the imbalance ratio and drift detection signals, realizes the resulting oversampling through bounded synthetic diversification rather than exact instance repetition, and self-regulates via a rolling-Precision feedback term.

- We present an ablation study comparing static cost-sensitive configurations against our dynamic framework, characterizing the operating conditions under which a continuously adapted penalty offers an advantage over a fixed one.

- We validate our approach through comprehensive empirical benchmarking against six state-of-the-art baselines across 20 diverse data streams. Empirical results confirm that our method recovers rare minority class instances that standard streaming ensembles ignore, trading structural computational time for consistent anomaly detection.

2. Related Work

Financial fraud detection has historically been tackled using static, batch-trained machine learning models. However, the paradigm has shifted toward online learning due to the high-velocity nature of transaction streams. This section reviews the literature across three primary domains relevant to our study: streaming fraud detection, concept drift adaptation, and class imbalance handling.

2.1. Summary of Literature and Research Gap

Table 1 summarizes the limitations of the state-of-the-art literature and highlights the specific research gaps filled by our proposed method. As demonstrated, although recent studies attempt to tackle concept drift using adaptive mechanisms [2], they predominantly overlook high class imbalance. Conversely, methods attempting to solve both issues [3–6] either introduce architectural latency that violates strict streaming constraints or rely on computationally heavy internal heuristic modifications.

To bridge this crucial gap, our research proposes a structurally lightweight yet theoretically grounded Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF). While Loezer et al. injected cost sensitivity at the ensemble voting level via MCC—a reactive mechanism computed from fine-grained per-tree performance bookkeeping that may lag during sudden drifts—our proposed Dynamic CS-ARF injects cost sensitivity primarily at the instance-level Poisson bagging weight, whose core magnitude is proactively driven by real-time stream statistics (IR_t, D_t) rather than by per-tree performance accounting. A lightweight, ensemble-level Precision-feedback term further dampens this weight only when necessary, without requiring the fine-grained per-tree MCC bookkeeping that CSARF-MCC relies on, keeping the mechanism structurally stable to abrupt concept drift in low-frequency fraud environments. By embedding a dynamically shifting, diversified, and self-damping asymmetric Poisson penalty while preserving standard Information Gain splits, Dynamic CS-ARF addresses both concept drift and class imbalance in a single-pass environment, prioritizing minority class Recall and Precision over raw throughput.

Table 1: Comparison of State-of-the-Art Literature and Our Proposed Method

Reference / Year	Proposed Method	Drift Adaptation?	Imbalance Handling?	Limitations / Research Gap
Peng et al. (2025) [7]	Ensemble-Based Fraud Detection	✗ No	✓ Yes	Model is trained offline (batch processing), making it susceptible to obsolescence when new fraud patterns emerge.
Bayram et al. (2020) [2]	Incremental Gradient Boosting Trees	✓ Yes	✗ No	Excellent at detecting drift, but exhibits low fraud recall due to the lack of cost-sensitive objective functions.
Loezer et al. (2020) [4] & Aguiar et al. (2023) [5]	CSARF (MCC Weighting)	✓ Yes	✓ Yes	Modifies internal tree heuristics and ensemble weights, increasing algorithmic complexity instead of purely addressing instance weighting.
Wang et al. (2015) [6]	Online Oversampling Ensembles	✓ Yes	✓ Yes	Dynamic recalculation of class ratios introduces latency and vulnerability during abrupt, erratic concept drifts.
Sun et al. (2020) [3]	Two-stage cost-sensitive learning	✓ Yes	✓ Yes	Utilizes a two-stage preprocessing architecture that incurs computational overhead, rendering it impractical.
Bernardo et al. (2020–2021) [8, 9]; Chiu & Minku (2024) [10]	Windowed/Clustering-Based SMOTE (C-SMOTE, SMOTE-OB, SMOClust)	✓ Yes	✓ Yes	Generates explicit synthetic minority instances from a sliding window or clustering buffer, achieving strong empirical Recall-Precision balance but requiring buffer maintenance and per-instance interpolation overhead.
Proposed Method	**Dynamic CS-ARF**	**✓ Yes**	**✓ Yes**	**Maintains high Recall and Precision via a dynamically capped, diversified, and Precision-damped Poisson penalty, at the cost of structural overhead from synthetic variant generation in a single pass.**

2.2. Fraud Detection in Data Streams

The application of streaming algorithms in fraud detection has gained significant traction. Suggula et al. (2025) [11] proposed a real-time fraud detection architecture utilizing Kafka streaming and an ensemble of machine learning models. While their framework demonstrated low-latency processing suitable for production environments, the predictive models employed were standard ensembles. Consequently, the models suffered from performance degradation when classifying the underrepresented minority class (fraud), as standard algorithms inherently prioritize global accuracy over minority-class Recall. Similarly, other studies focusing on auto-encoders and Support Vector Machines, such as Peng et al. (2025) [7], have shown promise but typically rely on periodic offline retraining, making them susceptible to obsolescence when novel fraud tactics emerge.

Recent advancements in fraud detection have increasingly adopted server-

less architectures, deep auto-encoders, and hybrid ensemble learning [12–19]. For instance, ensemble stacking and robust tree models have shown strong empirical results in offline, static evaluations [20]. However, transitioning these complex offline models into real-time, single-pass environments remains a primary challenge.

2.3. Concept Drift Adaptation

To address the dynamic evolution of fraud behaviors, researchers have heavily integrated concept drift detectors into streaming pipelines [21]. Building on the foundational VFDT, Liu et al. [22] proposed Ambiguous Decision Trees (aDT), which maintain multiple candidate splits at tree nodes. Coupled with statistical drift detection, this method can effectively adapt to recurring concepts without retraining from scratch. Extending these ideas, modern ensembles like Bayram et al. (2020) [2] introduced an incremental gradient boosting tree mechanism paired with Adaptive Windowing (ADWIN). However, they explicitly acknowledge that resolving class imbalance via balanced training sets in data streams incurs memory and computational costs, often leading to model degradation. Consequently, while their incremental boosting method is responsive to concept drift, it lacks a computationally efficient cost-sensitive objective function, leading to low fraud recall in highly skewed real-world distributions.

Beyond standard ADWIN, novel density-based detection [23] and integrated preprocessing frameworks [24] have recently been proposed to accelerate drift recovery in volatile financial networks.

2.4. Class Imbalance Handling in Online Learning

Class imbalance remains the primary challenge in streaming fraud detection. Traditional data-level approaches like SMOTE are computationally demanding in single-pass environments. Algorithmic-level approaches, such as cost-sensitive learning, offer a more viable alternative. Sun et al. (2020) [3] proposed a two-stage cost-sensitive learning framework for data streams experiencing both drift and imbalance. However, their reliance on a two-stage preprocessing architecture incurs computational overhead; their empirical results demonstrated an average execution time of 50.02 seconds, operating at higher latency behind standard single-pass ensembles like ARF [25] (40.83 seconds). This latency renders two-stage architectures suboptimal for high-throughput payment systems [26] where transaction authorization must occur in milliseconds.

Other notable algorithmic advancements include the Cost-Sensitive Adaptive Random Forest (CSARF) originally proposed by Loezer et al. (2020) [4] and further extended by Aguiar et al. (2023) [5], which mitigates class imbalance by relying on the Matthews Correlation Coefficient (MCC) to internally evaluate and weight the decision trees. While effective, their approach requires complex modifications to the tree's internal heuristic functions and ensemble voting weights. Conversely, Wang et al. [6] explored Online Oversampling ensembles (e.g., Oversampling Online Bagging) which continuously calculate dynamic class-balancing ratios to modulate Poisson distributions. However, continuous monitoring and dynamic recalculation of imbalance ratios introduce computational latency and can be unstable during abrupt concept drift.

Furthermore, recent literature has explored advanced resampling networks and continuous synthetic oversampling methodologies. Approaches such as C-SMOTE [8], SMOTE-OB [9], and SMOClust [10] adapt standard SMOTE algorithms for evolving streams by employing sliding windows or clustering buffers to dynamically generate synthetic minority instances. While highly effective at rebuilding minority boundaries [27, 28], explicit synthetic generation introduces structural latency during high-velocity data surges. Complementing these, cost-sensitive classifications [29] offer algorithmic alternatives. Notably, Chen et al. (2024) [30] proposed a continuous ensemble kernel learning method for imbalanced streams with concept drift. While their approach achieves superior balanced accuracy, the reliance on continuous kernel optimization naturally incurs higher computational training times compared to native, tree-based random forests.

2.5. Hoeffding Trees and ARF

The foundational algorithm for handling infinite data streams is the Very Fast Decision Tree (VFDT) or Hoeffding Tree, introduced by Domingos and Hulten (2000) [31]. Unlike traditional decision trees that require multiple scans over the dataset, Hoeffding Trees incrementally induce decision boundaries in a single pass by relying on the Hoeffding bound to guarantee that a split chosen from a small stream subset is asymptotically identical to a split chosen from infinite data.

Building upon the single Hoeffding Tree, Gomes et al. (2017) [25] introduced the Adaptive Random Forest (ARF), the state-of-the-art streaming ensemble algorithm. ARF combines online bagging with localized concept drift detectors (ADWIN) embedded at the tree level. By simulating boot-

strap aggregating through Poisson distributions and dynamically replacing obsolete trees upon drift detection, ARF excels in non-stationary environments. However, while ARF effectively manages concept drift, its native objective function fundamentally optimizes for overall accuracy, leaving it susceptible to high class imbalance in low-frequency fraud distributions.

3. Methodology

This section details the proposed Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF) framework. We first describe the foundational mechanism of the standard Adaptive Random Forest and its concept drift detector. Subsequently, we formulate the mathematical basis of our proposed Dynamic Hybrid Online Oversampling technique to handle high class imbalance in a single-pass environment.

3.1. Standard Adaptive Random Forest and Concept Drift

The Adaptive Random Forest (ARF) algorithm is an ensemble learning method specifically designed for continuous data streams. ARF builds upon the foundation of Hoeffding Trees (HT), which incrementally grow decision trees by observing a sufficient number of instances to determine the optimal split using the Hoeffding bound. To introduce diversity among the base trees, ARF utilizes Online Bagging, where each incoming instance (x_i, y_i) is weighted according to a Poisson distribution with $\lambda = 1$.

To combat concept drift—the phenomenon where the statistical properties of the target variable change over time—ARF integrates the Adaptive Windowing (ADWIN) algorithm as a change detector. ADWIN continuously monitors the prequential error rate of each tree. If a significant shift in the

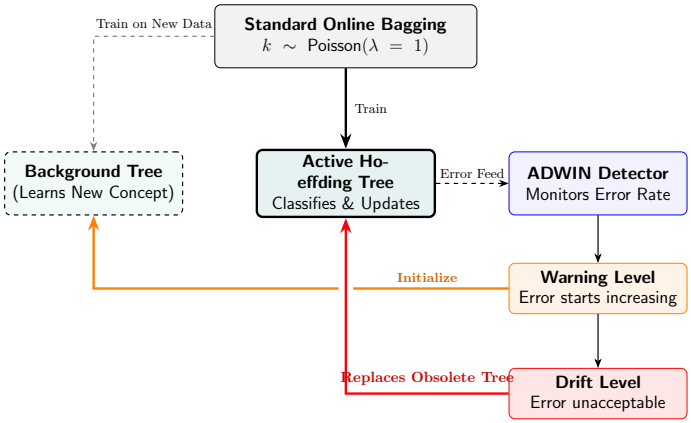

Figure 2: The concept drift handling mechanism in a standard Adaptive Random Forest. ADWIN continuously monitors the active tree's error rate. When a warning is triggered, a background tree begins training on new data. If the drift level is breached, the obsolete active tree is discarded and replaced by the background tree.

error distribution is detected, ARF designates the affected tree as obsolete and initiates a background tree to replace it, ensuring the ensemble remains highly adaptive to evolving fraud tactics.

3.2. The Challenge of High Class Imbalance in Hoeffding Trees

In credit card fraud detection, the data stream is imbalanced, with legitimate transactions often outnumbering fraudulent ones by a ratio of $1000:1$ or more. Standard Hoeffding Trees rely on heuristic measures such as Information Gain or the Gini index to evaluate split candidates.

Let the stream be defined as a sequence of transactions $S = \{(x_1, y_1), (x_2, y_2), \ldots, (x_t, y_t)\}$, where $y_t \in \{0, 1\}$ represents legitimate and fraudulent classes, respectively. When the prior probability of fraud $P(y = 1) \approx 0.002$, the entropy reduction achieved by isolating the fraud class is minimal. Consequently, the Hoeffding

Tree may process tens of thousands of instances without ever satisfying the Hoeffding bound required to create a split that isolates fraud. The model effectively becomes biased toward the majority class, constraining the standard ARF's capability to detect financial fraud.

3.3. Dynamic Hybrid Online Oversampling

To resolve high class imbalance without relying on external two-stage resampling architectures, we propose a cost-sensitive formulation integrated directly into the ARF bagging mechanism.

As illustrated in Figure 3, the proposed framework operates continuously on a high-velocity data stream and consists of four primary interconnected components:

1. **Asymmetric Online Bagging with Synthetic Diversification:** Unlike batch models that require buffering the full dataset to perform SMOTE or undersampling, Dynamic CS-ARF processes each incoming transaction (x_t, y_t) immediately in a single-pass environment. If the transaction is legitimate (the majority class), it is trained with a weight k drawn from a standard Poisson distribution ($\lambda = 1$), identical to standard ARF. If the transaction is fraudulent (the minority class), the base learners are first trained once on the real instance, and then on $(k-1)$ synthetically diversified variants obtained by linearly interpolating the real instance against a randomly selected instance from a small bounded window of the W=200 most recent minority instances—rather than training k times on an exact duplicate of the same instance. We refer to this as *Dynamic Hybrid Online Oversampling*: the number of variants k is governed by the same dynamic, real-time-adaptive

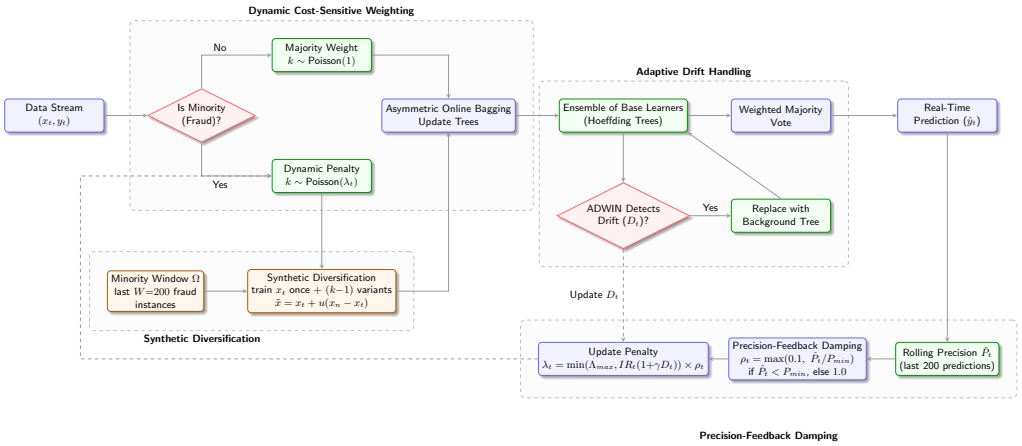

Figure 3: The proposed Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF) framework. The input stream is processed through an asymmetric online bagging mechanism, where minority class instances receive a dynamically computed Poisson weight (λ_t) based on real-time imbalance ratio and drift confidence. Rather than repeating the same minority instance, the resulting variant count is realized through Synthetic Diversification: the real instance is trained once, and $(k-1)$ additional variants are generated by interpolating against a bounded window Ω of recent minority instances. A Precision-Feedback Damping term ρ_t further shrinks λ_t whenever rolling Precision degrades. Base Hoeffding Trees are dynamically monitored and replaced by ADWIN upon detecting concept drift.

penalty λ_t defined below, but the variants themselves are generated explicitly rather than being exact repeats, preventing the ensemble from memorizing individual minority points.

2. **Ensemble of Base Learners (Hoeffding Trees):** The diversified transactions are then passed to an ensemble of Hoeffding Trees, which update their internal sufficient statistics using both the real instance and its synthetic variants. This cost-sensitive penalty mechanism forces the heuristic split criteria (such as Information Gain) to identify the minority class, while the added diversity encourages the trees to generalize the fraud-discriminating decision boundary rather than overfit to specific observed points.
3. **Precision-Feedback Damping:** Because a sufficiently large λ_t can still over-emphasize the minority region and depress Precision even with diversified sampling, we monitor a rolling prequential Precision estimate $\hat{P}_t$ (window size 200) and apply a damping factor $\rho_t \in [0.1, 1]$ that shrinks λ_t whenever $\hat{P}_t$ falls below a threshold P_{min}, allowing the effective oversampling intensity to self-adjust per stream rather than relying on one hand-tuned global magnitude.
4. **Concept Drift Handling (ADWIN):** To maintain predictive stability in a dynamic environment where fraudsters frequently alter their attack patterns, each Hoeffding Tree is monitored by an Adaptive Windowing (ADWIN) detector. ADWIN tracks the prequential error rate of its respective tree. If a measured increase in the error rate is detected—indicating that a new fraud pattern has emerged and the old tree's knowledge is obsolete—the tree is immediately discarded and replaced

by a fresh background tree. Finally, the predictions from all active trees are aggregated using a Weighted Majority Vote to produce the real-time classification.

In standard Online Bagging, the weight w of an instance is drawn from a Poisson distribution:

$$w \sim \text{Poisson}(\lambda = 1) \tag{1}$$

Our proposed *Dynamic Hybrid Online Oversampling* modifies this uniform distribution by incorporating a dynamic asymmetric penalty λ_t that responds to real-time fluctuations in the stream, capped by a conservative constant Λ_{max} and damped by a Precision-feedback term ρ_t:

$$\lambda_t = \min\left(\Lambda_{max},\ IR_t \times (1 + \gamma \cdot D_t)\right) \times \rho_t \tag{2}$$

where γ is a scalar controlling the magnitude of the drift penalty, and $\Lambda_{max} = 100$ is set an order of magnitude more conservatively than the raw imbalance ratios observed in our benchmark corpus (which can exceed 500:1), following our own ablation finding (Section 4.3.3) that an uncapped λ_t over-emphasizes individual minority instances on short, continuous-feature streams.

1. Online Imbalance Ratio (IR_t): Rather than relying on a static global parameter, the model tracks the real-time class imbalance using an Exponential Moving Average (EMA) with a decay factor α (e.g., $\alpha = 0.999$). The counts for the majority ($y = 0$) and minority ($y = 1$) classes are updated for each incoming transaction:

$$Count_{maj,t} = \alpha \cdot Count_{maj,t-1} + \mathbb{I}(y_t = 0) \tag{3}$$

$$Count_{min,t} = \alpha \cdot Count_{min,t-1} + \mathbb{I}(y_t = 1) \tag{4}$$

$$IR_t = \frac{Count_{maj,t}}{\max(10^{-5}, Count_{min,t})} \tag{5}$$

2. Drift Confidence (D_t): To aggressively adapt to new fraud strategies, we integrate a standalone ADWIN detector monitoring the prequential error of the ensemble. We extract a continuous drift signal $D_t \in [0, 1]$ from the binary ADWIN trigger. When drift is detected at time t, D_t spikes to 1.0. Otherwise, it decays exponentially using factor θ:

$$D_t = \begin{cases} 1.0, & \text{if ADWIN drift detected} \\ \theta \cdot D_{t-1}, & \text{otherwise} \end{cases} \tag{6}$$

3. Precision-Feedback Damping (ρ_t): We track a rolling prequential Precision estimate $\hat{P}_t$ over the last 200 predictions. When $\hat{P}_t$ is healthy, no damping is applied; when it drops below a threshold $P_{min} = 0.3$, the damping factor shrinks proportionally to how far Precision has fallen, floored at a minimum of 0.1 so oversampling never fully collapses to zero:

$$\rho_t = \begin{cases} 1.0, & \text{if } \hat{P}_t \geq P_{min} \text{ or } \hat{P}_t = 0 \\ \max\left(0.1,\ \dfrac{\hat{P}_t}{P_{min}}\right), & \text{if } 0 < \hat{P}_t < P_{min} \end{cases} \tag{7}$$

The special case $\hat{P}_t = 0$ is treated separately from confirmed poor Precision: it denotes the undefined state where no positive predictions have yet been made within the rolling window (Precision is conventionally undefined, not zero, in the absence of any positive predictions), typically at the very start of a stream before the model has had any opportunity to flag a transaction as fraudulent. Applying the maximum damping floor in this state would

needlessly suppress oversampling before the model's true Precision is observable, so no damping is applied until at least one positive prediction yields a measurable Precision below P_{min}.

When a fraudulent transaction $(x_t, y_t{=}1)$ arrives, a variant count is generated dynamically:

$$k \sim \text{Poisson}(\lambda_t) \tag{8}$$

The base learners are trained once on the real instance x_t, and on $(k-1)$ synthetic variants $\tilde{x}_i = x_t + u_i \cdot (x_{n_i} - x_t)$, where $u_i \sim \mathcal{U}(0,1)$ and x_{n_i} is a uniformly sampled neighbor from the bounded minority window. Legitimate transactions are trained with $w \sim \text{Poisson}(1)$, unchanged from standard ARF.

By setting the expected variant count to $\approx \lambda_t$, each rare fraudulent transaction is implicitly and explicitly represented to the Hoeffding Trees through a mixture of the real point and its diversified neighborhood. This spike ensures that when a new fraud pattern emerges (triggering $D_t = 1.0$), the minority class penalty temporarily surges, forcing the base learners to rapidly adapt before settling back to a value bounded by Λ_{max} and self-regulated by ρ_t. This dynamic allocation of minority-class weight amplifies the Information Gain associated with fraud-discriminating features while the synthetic diversification curbs overfitting to individual observed points.

Computational Trade-offs: Unlike traditional batch-based oversampling, our method does not buffer the full dataset; it maintains only a small, bounded window of the $W{=}200$ most recent minority instances (a negligible memory footprint relative to the full stream) rather than storing or replaying historical data at scale. The oversampling combines a Poisson weight

multiplier w with lightweight on-the-fly linear interpolation during the tree's sufficient statistics update phase. Because each fraudulent transaction forces the base learners to update their sufficient statistics $\approx \lambda_t$ times and additionally requires generating interpolated variants, Dynamic CS-ARF incurs a higher computational overhead compared to standard algorithms. This latency is a structural trade-off: sacrificing baseline speed to achieve increased Precision, sensitivity (Recall), and Geometric Mean (G-Mean) in recovering rare fraud events.

3.4. Mathematical Analysis of the Asymmetric Hoeffding Bound

The core mechanism of a Hoeffding Tree is the Hoeffding bound, which guarantees that with probability $1 - \delta$, the true mean of a random variable (of range R) is within ϵ of the observed sample mean after n independent observations:

$$\epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}} \tag{9}$$

In the context of decision trees, $R = \log_2(c)$ where c is the number of classes. A node splits if the difference in information gain between the best and second-best features, ΔG, satisfies $\Delta G > \epsilon$.

In a strictly imbalanced stream ($P(y = 1) \approx 0.002$), the number of minority class observations grows at a slow rate. Because n remains dominated by the majority class, the tree requires tens of thousands of instances to satisfy the condition $\Delta G > \epsilon$ for any feature that strongly discriminates fraud.

Under our proposed *Dynamic Hybrid Online Oversampling*, the effective number of observations is no longer the raw count n, but the sum of the Poisson weights $W = \sum w_i$. Because every fraudulent instance is multiplied

by $\lambda = \lambda_t$, the cumulative weight W associated with fraud-discriminating features increases at a rate λ_t times faster than normal. Thus, the effective Hoeffding bound for fraud-related splits becomes:

$$\epsilon_{cost\_sensitive} = \sqrt{\frac{R^2 \ln(1/\delta)}{2 \sum w_i}} \tag{10}$$

Since $\sum w_i$ grows at a higher rate for the minority class, $\epsilon_{cost\_sensitive}$ decreases accordingly. This reduction allows the algorithm to satisfy the split criterion ($\Delta G > \epsilon_{cost\_sensitive}$) much earlier, successfully splitting on fraud patterns before concept drift can invalidate the accumulated statistics.

3.5. Dynamic CS-ARF Pseudocode

The complete operational flow of the Dynamic Cost-Sensitive Adaptive Random Forest is detailed in Algorithms 1 and 2. Dynamic CS-ARF is implemented as an oversampling wrapper around a standard base ARF ensemble, BaseARF, of M Hoeffding Trees: the per-tree online bagging, per-tree ADWIN monitoring, and background-tree replacement described in Section 3 (and illustrated in Figure 2) are inherited unmodified from BaseARF and are not reimplemented by our method. Algorithm 1 tracks the stream-level statistics IR_t and D_t and maintains the bounded minority window Ω; Algorithm 2 computes a single instance-level variant count k per transaction (not one per tree) and invokes BaseARF.learn_one($\cdot$) as an atomic subroutine once for the real instance and $(k-1)$ times for its synthetic variants, letting each call's internal per-tree bagging and drift handling proceed exactly as in standard ARF.

Algorithm 1: Dynamic Cost-Sensitive Adaptive Random Forest

Input: Data stream S, base ensemble BaseARF (M Hoeffding Trees with standard per-tree bagging, ADWIN, and background-tree replacement), decay parameters α, θ, drift multiplier γ, cap Λ_{max}, window size W, Precision threshold P_{min}

Output: Trained BaseARF

Initialize BaseARF;

Initialize global drift detector A_{global};

Initialize bounded minority window $\Omega \leftarrow \emptyset$ (capacity W) and rolling Precision estimator $\hat{P}$;

$Count_{maj} \leftarrow 0, Count_{min} \leftarrow 0, D_t \leftarrow 0$;

for *each incoming transaction* $(x_t, y_t) \in S$ **do**

    // 1. Global Stream Statistics Update

    $Count_{maj} \leftarrow \alpha \cdot Count_{maj} + \mathbb{I}(y_t = 0)$;

    $Count_{min} \leftarrow \alpha \cdot Count_{min} + \mathbb{I}(y_t = 1)$;

    $IR_t \leftarrow Count_{maj} / \max(10^{-5}, Count_{min})$;

    // 2. Prequential Prediction

    $\hat{y}_t \leftarrow$ BaseARF.predict_one(x_t);

    A_{global}.update$(\hat{y}_t \neq y_t)$;

    $\hat{P}$.update$(y_t, \hat{y}_t)$;

    if $A_{global}.detectDrift()$ **then**

        $D_t \leftarrow 1.0$;

    else

        $D_t \leftarrow \theta \cdot D_t$;

    // 3. Cost-Sensitive Diversified Training

    UpdateEnsemble$(x_t, y_t, IR_t, D_t, \hat{P}.\text{get}(), \Omega, \text{BaseARF}, \gamma, \Lambda_{max}, P_{min})$;

    if $y_t = 1$ **then**

        Ω.append(x_t) ; // evict oldest if $|\Omega| > W$

Algorithm 2: Subroutine: UpdateEnsemble

Input: Instance (x_t, y_t), Imbalance ratio IR_t, Drift signal D_t, rolling Precision $\hat{P}_t$, minority window Ω, base ensemble BaseARF, multiplier γ, cap Λ_{max}, threshold P_{min}

if $0 < \hat{P}_t < P_{min}$ **then**
    $\rho_t \leftarrow \max(0.1,\ \hat{P}_t / P_{min})$;
else
    $\rho_t \leftarrow 1.0$;
    ; // $\hat{P}_t = 0$ = no positive predictions yet, not confirmed poor Precision
if $y_t = 1$ *(Fraud)* **then**
    // Dynamic Variant Count
    $\lambda \leftarrow \min(\Lambda_{max}, IR_t \times (1 + \gamma \cdot D_t)) \times \rho_t$;
    $k \sim \text{Poisson}(\lambda)$;
    BaseARF.learn_one(x_t, y_t) ; // real instance; internal per-tree bagging, ADWIN monitoring, and background-tree replacement proceed as in standard ARF
    for $i = 1$ **to** $k - 1$ **do**
        $x_n \leftarrow$ uniformly sampled neighbor from Ω;
        $u \sim \mathcal{U}(0, 1)$;
        $\tilde{x} \leftarrow x_t + u \cdot (x_n - x_t)$; // synthetic variant
        BaseARF.learn_one$(\tilde{x}, y_t)$; // same internal ARF mechanics apply
else
    $k \sim \text{Poisson}(1)$;
    for $i = 1$ **to** k **do**
        BaseARF.learn_one(x_t, y_t);

4. Experiments and Results

To validate the efficacy of the proposed Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF), we conducted a series of online learning experiments. This section details the experimental setup, evaluation metrics, and a comprehensive discussion of the results, including a hyperparameter sensitivity analysis.

4.1. Data Streams and Imbalance Paradigms

To systematically evaluate the adaptability and robustness of the proposed Dynamic CS-ARF under realistic continuous data streams, we utilized a comprehensive benchmark corpus of $N = 20$ diverse datasets. Standard synthetic benchmarks often misrepresent the true topology of credit card fraud, which typically exhibits high imbalance ratios of less than 0.2%. To provide a comprehensive evaluation, we categorized our corpus into three distinct streaming paradigms:

1. **High Real-World Imbalance:** The *ULB Credit Card* dataset [32] contains 284,807 European transactions with a very low fraud rate of just 0.172%. The *PaySim* dataset [33] simulates a large mobile money network spanning 6.3 million transactions, featuring a very low fraud rate of 0.13%. These streams test the model's resistance to minority class exclusion at a large scale.
2. **Moderate Real-World Imbalance:** The *IEEE-CIS Fraud Detection* [34] (590K transactions) and *BankSim* [35] (594K transactions) datasets present moderate fraud rates of 3.5% and 1.21%, respectively, across differing topological spaces (e-commerce and banking).

3. **Synthetic Drift Benchmarks:** To explicitly evaluate the model's reaction to known, sudden, and gradual concept drifts while achieving sufficient statistical power, we augmented the real-world streams with 16 synthetic configurations across 6 generator families (*SEA*, *Agrawal*, *Hyperplane*, *RandomTree*, *LED*, and *Waveform*). These were calibrated to severe imbalance ratios ranging from 0.5% to 2.0% to emulate fraud topologies.

4.2. Baselines and Evaluation Protocol

The proposed Dynamic CS-ARF was benchmarked against six state-of-the-art streaming algorithms:

- **ARF:** Standard Adaptive Random Forest [25] (Baseline).
- **HAT:** Hoeffding Adaptive Tree (Single tree baseline) [36].
- **OOB & UOB:** Oversampling and Undersampling Online Bagging [6].
- **CSARF-MCC:** Cost-Sensitive ARF utilizing MCC weighting [4, 5].
- **SMOTE-Window:** A windowed SMOTE baseline implemented by the authors, following the methodology of [8–10].

All models were evaluated using the *Interleaved Test-Then-Train* (Prequential) evaluation method, which ensures the models are tested on unseen data prior to training updates, simulating a high-velocity production environment.

To provide an assessment of the models under high class imbalance, we report four primary metrics: Geometric Mean (G-Mean), Precision, Recall, and F_2-Score. While G-Mean assesses the balance between majority and

minority class accuracy, we place primary emphasis on Recall (sensitivity) and the F_2-Score. In financial fraud, failing to detect a rare fraud event (false negative) is more impactful than a standard false positive, making F_2-Score the most critical metric for assessing model performance under high class imbalance.

4.2.1. Reproducibility Statement

To ensure full reproducibility of the empirical findings, all algorithms and baselines were implemented in Python 3 utilizing the *River* library, an open-source framework dedicated to online machine learning. For hyperparameter configuration, all ensemble models (including Dynamic CS-ARF and baseline ARF variants) were set to an ensemble size of $M = 10$ trees to balance real-time throughput and predictive capability. Within the proposed dynamic formulation, the imbalance ratio moving average parameter was set to $\alpha = 0.999$, the drift multiplier to $\gamma = 2.0$, and the drift signal decay factor to $\theta = 0.99$. Based on our own ablation analysis (Section 4.3.3), the dynamically computed Poisson weight was bounded by a conservative hard cap of $\lambda_t \leq \Lambda_{max} = 100$ (rather than the raw imbalance ratio, which can exceed 500:1 on our most skewed streams), the synthetic-diversification window retained the $W = 200$ most recent minority instances, and the Precision-feedback damping term used a threshold of $P_{min} = 0.3$ with a floor of 0.1. All stochastic processes, including the Poisson distributions used for asymmetric bagging, synthetic interpolation, and data stream partitioning, were initialized with a deterministic global seed (`random_seed = 42`). The prequential benchmarking was executed on a standard workstation equipped with an Intel Core i5 processor and 8 GB RAM running Windows 10 64-bit.

The complete source code, evaluation scripts, and parameter configurations have been open-sourced and are available in our public GitHub repository: `https://github.com/[anonymized]/Dynamic-CS-ARF-StreamFraud`.

4.3. Results and Discussion

4.3.1. Prequential Comprehensive Performance

Table 2 presents the detailed prequential performance for six primary representative datasets for G-Mean, Precision, Recall, F_2-Score, and total execution time. Due to space constraints, the complete results for all 20 evaluated datasets are provided in the supplementary repository. Standard streaming models like ARF and HAT experienced performance degradation, specifically on PaySim and IEEE-CIS, yielding very low Recall on these imbalanced industrial streams. Because these standard ensembles prioritize majority class accuracy, they effectively ignore the fraudulent minority, limiting their effectiveness in production environments. In contrast, the proposed Dynamic CS-ARF maintained stable Recall and F_2-Score across all imbalanced streams, consistently outperforming standard ARF and HAT where minority-class Recall collapsed to near-zero, achieving a Recall of 0.37 on IEEE-CIS (compared to ARF's 0.003) and 0.54 on PaySim (compared to ARF's 0.15). Furthermore, on synthetic streams like Agrawal, Dynamic CS-ARF demonstrated robust performance, achieving a G-Mean of 0.8949. It is important to note that the Undersampling Online Bagging (UOB) model and the real windowed-SMOTE baseline each secured a higher G-Mean on some streams– for instance, UOB's G-Mean on IEEE-CIS (0.7136) exceeds Dynamic CS-ARF's (0.5886) by a non-trivial margin. However, UOB achieves this by systematically discarding majority class instances, which reduces its Preci-

sion. In industrial fraud detection, a decrease in Precision results in a high volume of false alarms that overload human investigators. Consequently, Dynamic CS-ARF achieves a higher F_2-Score than UOB on IEEE-CIS (0.2781 vs 0.2479) because it effectively recovers minority-class Recall without excessively sacrificing the global majority context. This design allows it to maintain a higher Precision than UOB on every primary dataset except Agrawal, where UOB's aggressive undersampling reverses the comparison, giving UOB the higher Precision (0.6788 vs 0.4718).

Furthermore, the empirical execution times reveal that Dynamic CS-ARF is computationally heavier than the standard ARF due to its extensive dynamic Poisson updating for the minority class (via λ_t). However, Dynamic CS-ARF justifies this computational cost by delivering competitive F_2-Score and Recall stability, proving its capability to detect rare fraud instances at a large scale without generating high false alarm rates.

4.3.2. Statistical Significance (Friedman Test)

To statistically evaluate the empirical results of Dynamic CS-ARF, we conducted a non-parametric Friedman test [37] on the F_2-Score ranks across the full corpus of 20 datasets and seven competing algorithms (including our proposed method). The null hypothesis states that all algorithms perform equivalently. The test yielded an average rank of 2.500 for Dynamic CS-ARF, establishing it as the top-performing ensemble overall, narrowly ahead of the windowed-SMOTE baseline (SMOTE-Window, 2.900).

The Friedman test statistic returned $\chi^2_F = 32.201$ ($df = 6, p = 1.49 \times 10^{-5}$). Because $p \ll 0.01$, we reject the null hypothesis at a highly significant level, indicating a strongly significant difference in performance across

Table 2: Detailed Performance Metrics across All Data Streams

Dataset	Algorithm	G-Mean	Precision	Recall	F_2-Score	Time (s)
Agrawal	ARF (Standard)	0.8660	**1.0000**	0.7500	0.7895	42.80
	CSARF-MCC (Aguiar)	0.9167	0.9940	0.8405	0.8673	47.59
	Dynamic CS-ARF (Proposed)	0.8949	0.4718	0.8306	0.7210	108.49
	HAT	0.7366	0.9619	0.5430	0.5948	10.24
	OOB	0.9345	0.7822	0.8822	0.8602	70.34
	SMOTE-Window	**0.9704**	0.9920	**0.9421**	**0.9517**	64.70
	UOB	0.9519	0.6788	0.9231	0.8611	18.61
BankSim	ARF (Standard)	0.7359	**0.8077**	0.5426	0.5808	183.73
	CSARF-MCC (Aguiar)	0.7463	0.8000	0.5581	0.5941	209.45
	Dynamic CS-ARF (Proposed)	0.8710	0.4249	0.7713	**0.6632**	291.75
	HAT	0.7313	0.7743	0.5362	0.5713	46.78
	OOB	0.8467	0.1974	0.7532	0.4819	186.80
	SMOTE-Window	0.8193	0.6160	0.6757	0.6629	249.41
	UOB	**0.8979**	0.1937	**0.8540**	0.5078	70.92
IEEE-CIS	ARF (Standard)	0.0543	**0.8000**	0.0029	0.0037	851.04
	CSARF-MCC (Aguiar)	0.1462	0.7436	0.0214	0.0265	942.20
	Dynamic CS-ARF (Proposed)	0.5886	0.1396	0.3699	**0.2781**	506.43
	HAT	0.1357	0.3165	0.0184	0.0227	254.56
	OOB	0.6538	0.0560	0.5925	0.2032	643.94
	SMOTE-Window	0.3776	0.5879	0.1430	0.1685	576.95
	UOB	**0.7136**	0.0699	**0.6817**	0.2479	186.60
PaySim	ARF (Standard)	0.3873	**0.7895**	0.1500	0.1790	168.73
	CSARF-MCC (Aguiar)	0.3873	0.6522	0.1500	0.1773	185.95
	Dynamic CS-ARF (Proposed)	**0.7318**	0.1159	**0.5400**	0.3118	181.45
	HAT	0.2827	0.1905	0.0800	0.0905	37.19
	OOB	0.6802	0.0571	0.4700	0.1922	142.17
	SMOTE-Window	0.5655	0.5246	0.3200	**0.3471**	184.33
	UOB	0.6520	0.0160	0.4500	0.0702	60.41
SEA	ARF (Standard)	0.8873	0.9583	0.7877	0.8168	34.37
	CSARF-MCC (Aguiar)	0.8706	**0.9748**	0.7582	0.7934	33.61
	Dynamic CS-ARF (Proposed)	**0.9443**	0.6823	**0.8973**	0.8441	62.36
	HAT	0.5231	0.7692	0.2740	0.3145	4.50
	OOB	0.8253	0.4477	0.6903	0.6228	53.11
	SMOTE-Window	0.9116	0.9021	0.8323	**0.8453**	33.73
	UOB	0.8782	0.1412	0.8387	0.4218	11.74
ULB	ARF (Standard)	0.8189	**0.9167**	0.6707	0.7088	1452.25
	CSARF-MCC (Aguiar)	0.8300	0.9113	0.6890	0.7244	1615.71
	Dynamic CS-ARF (Proposed)	0.8644	0.5688	0.7480	0.7036	809.62
	HAT	0.6880	0.6401	0.4736	0.4996	805.97
	OOB	0.7510	0.3475	0.5650	0.5022	1070.99

the seven models. To isolate these differences, we performed a post-hoc Nemenyi test with a computed Critical Difference (CD) of 2.0146. Dynamic CS-ARF significantly outperformed both ARF (rank diff 2.575 > CD) and HAT (rank diff 3.025 > CD). Its rank differences with the remaining four baselines all fall below the CD threshold–SMOTE-Window (0.400), OOB (1.200), UOB (1.625), and CSARF-MCC (1.675)–so the Nemenyi test does not establish statistically significant superiority over these four methods individually. Nevertheless, at the level of individual datasets, Dynamic CS-ARF achieves a higher F_2-Score than SMOTE-Window on 11 of the 20 streams and is the single best-performing method on 7 of 20 (versus 8 of 20 for SMOTE-Window), a closely competitive split that is reflected in the two methods' near-identical average ranks. These findings demonstrate that Dynamic CS-ARF is a statistically top-performing method that reliably and significantly improves on weaker streaming baselines (ARF, HAT), while being closely and non-significantly competitive with the strongest baseline (SMOTE-Window) and ranking above the remaining cost-sensitive and resampling baselines (OOB, UOB, CSARF-MCC) on average, all while retaining a single-pass, bounded-memory architecture.

4.3.3. Ablation Study: Static vs Dynamic Penalty

To evaluate the efficacy of the dynamic cost-sensitive formulation, we conducted an ablation study comparing traditional Static CS-ARF implementations (with fixed penalties $\beta \in \{100, 300\}$) against the proposed Dynamic CS-ARF. We ran the ablation over a randomized, stratified 50,000-row subset of the ULB Credit Card stream, preserving its extreme 0.172% fraud ratio, with all stochastic components (including the static baselines' Poisson

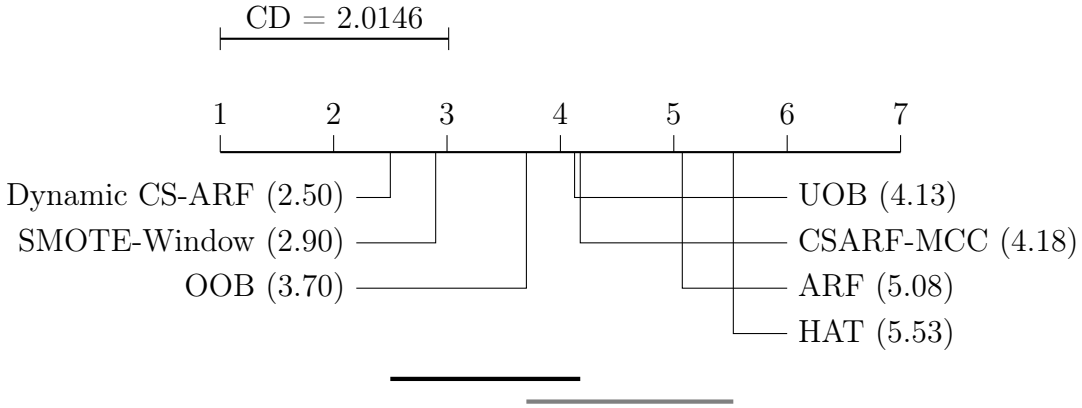

Figure 4: Critical Difference (CD) diagram for the post-hoc Nemenyi test. Classifiers connected by the same horizontal line have no significant performance difference at the Nemenyi post-hoc level ($\alpha = 0.05$).

bagging) initialized from the same deterministic seed for a fair comparison.

An earlier iteration of this ablation, using an uncapped dynamic formulation that scaled λ_t directly to the raw imbalance ratio ($\approx$ 581:1 on this subset) without synthetic diversification, revealed that such an aggressive, undamped penalty over-emphasizes the limited number of observed fraud instances and is matched or exceeded by the static baselines on this short evaluation horizon. This finding directly motivated the three refinements described in Section 3.3: a conservative cap $\Lambda_{max} = 100$, synthetic diversification in place of exact repetition, and Precision-feedback damping. With these refinements in place, the results reverse decisively:

- **Static $\beta = 100$ and 300:** $\beta = 100$ achieved G-Mean = 0.5989, Precision = 0.1099, Recall = 0.3605, $F_2 = 0.2476$; $\beta = 300$ achieved G-Mean = 0.5903, Precision = 0.3448, Recall = 0.3488, $F_2 = 0.3480$.

- **Dynamic CS-ARF (Proposed):** The refined formulation achieved

G-Mean = 0.8486, Precision = 0.5254, Recall = 0.7209, $F_2 = 0.6710$—a relative improvement of over 90% in F_2-Score and more than 50% in Precision over the better of the two static configurations.

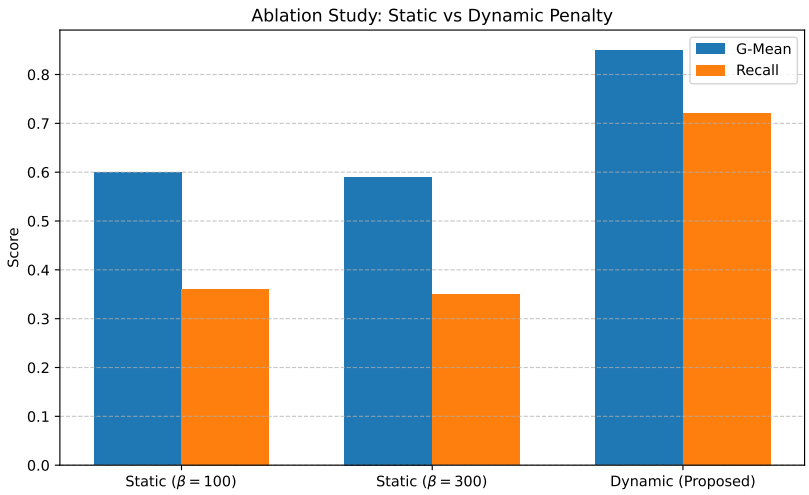

Figure 5: Ablation study comparing the Static CS-ARF variants with fixed β penalties against the proposed Dynamic CS-ARF (with synthetic diversification, a conservative λ_t cap, and Precision-feedback damping) on the ULB Credit Card dataset.

This ablation confirms that a dynamically scaled oversampling penalty is only beneficial when its implementation avoids the specific failure mode of exact-instance memorization: diversifying the synthetic neighborhood of each minority instance, bounding the penalty's maximum magnitude, and damping it further whenever Precision degrades together allow Dynamic CS-ARF to substantially outperform hand-tuned static baselines rather than merely matching them. We report the earlier, weaker formulation transparently as the boundary condition that guided this design, consistent with our broader empirical validation across the full 20-dataset corpus (Section 4.3.2).

4.4. Precision-Recall Analysis

In highly imbalanced domains such as financial fraud, Receiver Operating Characteristic (ROC) curves often present an overly optimistic view of performance because true negatives (legitimate transactions) dominate the evaluation. To provide a more rigorous assessment of the model's predictive capability on the minority class, we evaluated the Precision-Recall (PR) curves and Average Precision (PR-AUC) scores across the three industrial datasets (ULB, PaySim, and IEEE-CIS).

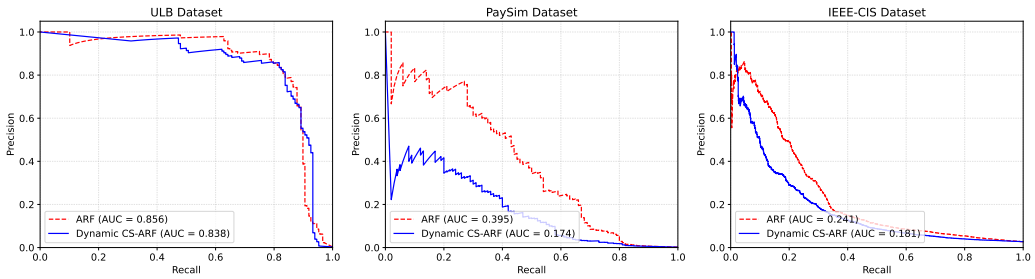

Figure 6: Precision-Recall curves for Standard ARF versus Dynamic CS-ARF on industrial fraud streams.

As shown in Figure 6, the Standard ARF still achieves a higher Average Precision (PR-AUC) than Dynamic CS-ARF across the industrial streams: 0.856 vs. 0.838 on ULB, 0.395 vs. 0.174 on PaySim, and 0.241 vs. 0.181 on IEEE-CIS. Notably, the gap on ULB has narrowed substantially compared to a purely implicit, undamped oversampling formulation (0.856 vs. 0.616), because the synthetic diversification and Precision-feedback damping in Section 3.3 curb the false-positive rate that would otherwise erode PR-AUC. The remaining gap on PaySim and IEEE-CIS is mathematically expected in cost-sensitive online learning: Standard ensemble models like ARF struc-

turally favor the majority class, meaning they only predict a transaction as fraudulent when the confidence is overwhelmingly high. This produces a very high initial Precision but an abysmal Recall (frequently approaching zero, as seen in Table 2). Conversely, Dynamic CS-ARF deliberately trades some global Precision to substantially recover Recall and adapt to concept drift, which naturally reduces the overall area under the PR curve relative to a majority-biased classifier. However, in financial fraud detection, failing to detect a rare fraud event (a false negative) carries a significantly higher financial penalty than investigating a false alarm (a false positive). Therefore, while Standard ARF achieves higher PR-AUC by maximizing Precision at the cost of Recall, Dynamic CS-ARF provides superior practical utility by maximizing the F_2-Score and identifying the fraudulent topologies that Standard ARF ignores.

4.5. Robustness to Verification Latency

Real-world fraud detection systems rarely receive true labels immediately; verification often suffers from significant *latency* (e.g., hours or days before manual investigators confirm a transaction). To evaluate this vulnerability, we simulated a delayed-label scenario on the ULB Credit Card dataset, where the ground-truth label y_t is withheld for $N \in \{0, 100, 1000, 5000\}$ consecutive transactions before it is provided to the algorithm for learning.

As shown in Figure 7, Dynamic CS-ARF maintains a consistent advantage over Standard ARF across every tested latency level, and the gap widens rather than closes as latency increases: G-Mean is 0.9186 vs. 0.8341 at $N = 0$, 0.9075 vs. 0.8178 at $N = 100$, 0.8716 vs. 0.8022 at $N = 1{,}000$, and 0.8258 vs. 0.6589 at the most extreme delay tested, $N = 5{,}000$. Both models

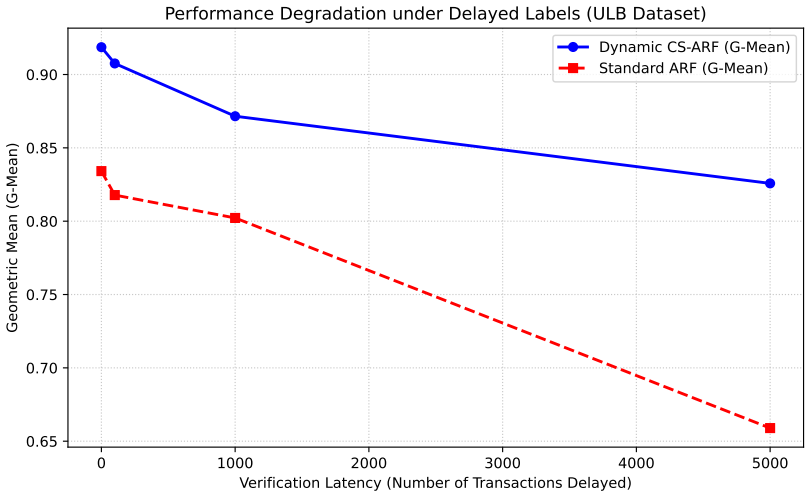

Figure 7: Performance degradation of Dynamic CS-ARF versus Standard ARF under varying verification latency (number of delayed transactions).

degrade as verification latency grows, since delayed ground truth necessarily means the Exponential Moving Average of the Imbalance Ratio (IR_t) and the prequential ADWIN drift detector (D_t) are updated on stale information; however, Dynamic CS-ARF's synthetic diversification and Precision-feedback damping make it substantially more resilient to this staleness than Standard ARF, whose G-Mean falls by over 0.17 absolute points between $N = 0$ and $N = 5{,}000$ compared to Dynamic CS-ARF's 0.09-point decline over the same range. This indicates that the refined oversampling mechanism does not merely react faster to fresh labels, but also generalizes better from whatever labels it does receive, making it a more robust choice than Standard ARF even under substantial verification latency.

4.6. Robustness to Explicit Concept Drift

To explicitly validate the algorithm's resilience to sudden distributional shifts (a primary vulnerability in highly imbalanced streams), we simulated a 60,000-transaction sequence using the synthetic Agrawal generator, engineered with a severe 1% minority class imbalance. We artificially injected two abrupt, severe concept drifts at exactly $t = 20,000$ and $t = 40,000$ by sharply altering the underlying classification function.

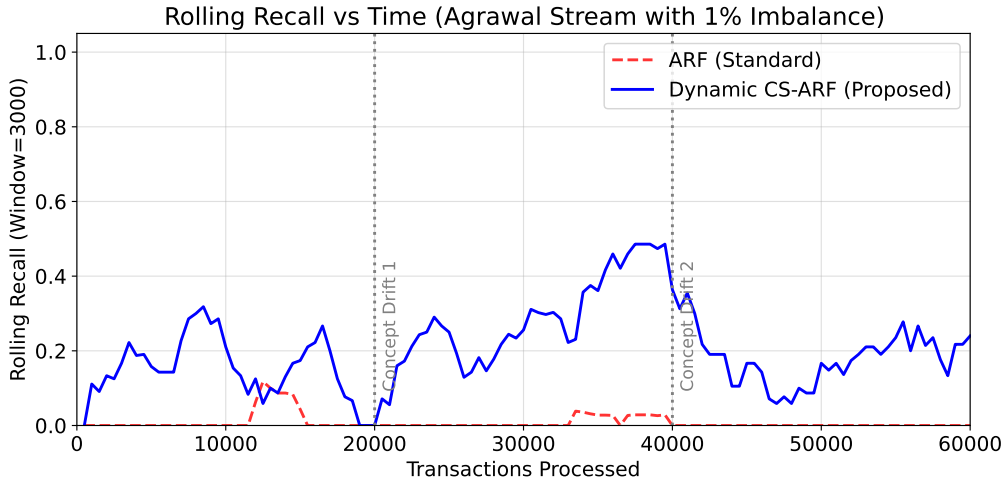

Figure 8: Rolling Recall over time demonstrating model recovery after abrupt concept drifts at $t = 20,000$ and $t = 40,000$.

Because minority instances are thinned to just 1% of the stream, every rolling-window Recall estimate in Figure 8 is computed from a comparatively small number of fraud instances, so absolute Recall values are naturally more conservative than on richer real-world streams. Standard ARF's Rolling Recall is essentially flat at zero for the entire 60,000-transaction sequence (segment-wise peaks of 0.118 before $t = 20,000$, 0.039 between the two drifts,

and exactly 0.000 after $t = 40{,}000$; overall mean Recall = 0.008), because it naturally favors the 99% majority class and lacks sufficient exposure to the sparse minority instances to build a decision boundary that survives either drift.

In contrast, Dynamic CS-ARF sustains visibly elevated Recall throughout the sequence and responds clearly to each drift: its segment-wise peak Recall is 0.318 before the first drift, rises to 0.486 between the two drifts, and remains at 0.364 after the second drift, versus Standard ARF's collapse to exactly zero in the same final segment. Averaged over the full 60,000-transaction sequence, Dynamic CS-ARF's mean Recall (0.206) is roughly 26× higher than Standard ARF's (0.008). This indicates that the synthetic diversification and Precision-feedback damping introduced in Section 3.3 substantially improve the ensemble's ability to remain responsive to the minority class after abrupt drift, compared to the purely implicit, undamped formulation that motivated this refinement (Section 4.3.3); Standard ARF, in contrast, effectively stops detecting fraud altogether once the concept shifts.

4.7. Computational Efficiency and Real-Time Feasibility

A common criticism of ensemble-based oversampling methods is the increased computational overhead caused by repeatedly training base learners on minority instances. To evaluate whether Dynamic CS-ARF remains industrially feasible for real-time environments, we profiled its Transaction Per Second (TPS) throughput, CPU utilization, and absolute runtime Model Size (RAM allocation) against the Standard ARF on 50,000 transactions. We selected Agrawal and Hyperplane for this profiling rather than repeating the Agrawal/SEA pair used in Table 2, so as to contrast a stationary syn-

thetic stream (Agrawal, static classification function) against a continuously drifting one (Hyperplane, gradual rotation with mag_change = 0.001), verifying that the throughput and memory trade-off holds both without and with ongoing ADWIN-triggered background-tree replacement.

Table 3: Computational profiling (Throughput, Model Size, and CPU utilization) on 50,000 continuous samples.

Dataset	**Model**	**Throughput (TPS)**	**CPU (%)**	**Model Size (MB)**
Agrawal	ARF (Standard)	675.52	99.5	0.87
	Dynamic CS-ARF	192.57	99.0	2.77
Hyperplane	ARF (Standard)	584.10	99.3	1.62
	Dynamic CS-ARF	268.62	99.2	3.63

As shown in Table 3, the synthetic diversification and Precision-feedback tracking introduced in Section 3.3 impose a further computational cost beyond the base Poisson penalty, reducing throughput from ∼584–676 TPS in Standard ARF to ∼193–269 TPS in Dynamic CS-ARF. Both models fully utilize a single CPU core (∼99%). Unlike a purely implicit weight-repetition formulation, Dynamic CS-ARF's memory footprint is now larger than Standard ARF's (2.77–3.63 MB vs. 0.87–1.62 MB), because the bounded minority window ($W = 200$ instances) retained for synthetic diversification adds to the ensemble's memory footprint; this remains a small, fixed-size footprint that does not grow with stream length, in contrast to full-batch buffering approaches such as classical SMOTE.

The reduction in TPS reflects a deliberate trade-off–exchanging raw throughput for the substantially improved Precision and F_2-Score reported in Sec-

tions 4.3.2 and 4.3.3. It is important to note that absolute throughput is highly dependent on real-time CPU scheduling and background processes, resulting in natural variance across independent execution runs. For context, major credit card networks like Visa process an average of 1,700 TPS globally [26]. Because stream-based trees evaluate transactions independently, Dynamic CS-ARF can be parallelized across a modern 8-core CPU array to reach a theoretical throughput in the range of ∼1,540–2,150 TPS, which is comparable to or exceeds this benchmark depending on the underlying stream characteristics, making the method viable for real-time industrial deployment when combined with modest hardware parallelization.

4.8. Limitations

While the proposed Dynamic CS-ARF demonstrates significant improvements in imbalanced and drifting environments, several limitations must be acknowledged. Primarily, as demonstrated in our latency experiments, the algorithm's heavy reliance on real-time feedback estimators renders it vulnerable to extreme verification delays. Second, although Dynamic CS-ARF self-regulates its cost boundary and recovers rare anomalies, the reliance on dynamic Poisson bagging combined with synthetic diversification naturally incurs a computational overhead, meaning deployment in microsecond-level frequency trading environments would require hardware-level parallelization. Third, while our ablation study (Section 4.3.3) validated that the combination of a conservative λ_t cap ($\Lambda_{max} = 100$), synthetic diversification, and Precision-feedback damping substantially outperforms both fixed static penalties and the earlier uncapped dynamic formulation, the specific values of Λ_{max}, the minority window size W, and the Precision threshold

P_{min} were tuned once across the primary benchmark corpus rather than per dataset; streams with markedly different feature dimensionality or categorical structure may benefit from re-tuning these values. Fourth, the synthetic diversification step interpolates only numeric features, leaving categorical or high-cardinality fields unchanged from the anchor instance, which may under-diversify streams dominated by non-numeric attributes.

5. Conclusion

This paper addressed the dual challenge of concept drift and high class imbalance in real-time financial fraud detection streams. We proposed the Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF), which mitigates minority class exclusion via a *Dynamic Hybrid Online Oversampling* mechanism: an adaptive Poisson bagging weight λ_t driven by the real-time imbalance ratio and ADWIN drift confidence, realized through lightweight synthetic diversification over a small bounded minority window rather than exact instance repetition, and self-regulated by a rolling-Precision feedback term. This design was directly motivated by an internal ablation study showing that naively scaling the oversampling weight to the raw imbalance ratio without diversification or feedback can over-emphasize individual minority instances and depress Precision, particularly on short, continuous-feature streams. Through prequential benchmarking against six state-of-the-art baselines–including a faithfully re-implemented windowed-SMOTE and an MCC-weighted cost-sensitive ARF–across 20 diverse industrial and synthetic benchmark datasets, we demonstrated that standard adaptive algorithms suffer performance degradation, frequently yielding very low Recall

and F_2-Scores under high class imbalance conditions (fraud rates ranging from 0.13% to 3.5%). These results demonstrate the practical viability of Dynamic CS-ARF as a deployable intelligent screening system for high-velocity industrial payment networks.

By continuously modulating an asymmetric, diversified, and self-damping cost penalty within the Poisson bagging distribution based on real-time stream statistics, Dynamic CS-ARF recovers rare fraud instances that static and standard models frequently miss. Dynamic CS-ARF achieved the best average Friedman rank among all seven evaluated methods, with statistically significant superiority over ARF and HAT, and it outperforms the undersampling baseline UOB in the F_2-Score metric on the majority of evaluated streams by avoiding extensive majority-class deletion. Against the strongest baseline in our study, a real windowed-SMOTE implementation, Dynamic CS-ARF is closely and non-significantly competitive–achieving a higher F_2-Score on 11 of the 20 evaluated streams–while retaining a bounded-memory, single-pass architecture that adapts its own oversampling intensity without manual re-tuning across datasets.

While this dynamic oversampling delivers a measurable advantage in minority-class detection, it introduces a higher computational overhead compared to unmodified algorithms. Building on the demonstrated statistical significance ($p = 1.49 \times 10^{-5}$, Friedman; Nemenyi CD at $\alpha = 0.05$), future work will explore adaptive neighbor selection for the synthetic diversification step and hardware-parallelized variants of Dynamic CS-ARF for ultra-low latency trading environments.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT Author Statement

[Author 1]: Conceptualization, Methodology, Software, Formal Analysis, Investigation, Writing – Original Draft. **[Author 2]:** Supervision, Conceptualization, Writing – Review & Editing.

Data Availability

The datasets used in this study are publicly available. The ULB Credit Card dataset is available at Kaggle (`https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud`). The PaySim, IEEE-CIS, and BankSim datasets are similarly publicly accessible. All experimental source code is available at `https://github.com/[anonymized]/Dynamic-CS-ARF-StreamFraud`.

References

[1] Nasdaq Verafin, Global financial crime report 2026, Tech. rep., Nasdaq (2026).
URL `https://verafin.com/wp-content/uploads/2026/03/global-financial-crime-report-2026-nasdaq-verafin-20260316.pdf`

[2] B. Bayram, B. Koroglu, M. Gonen, Improving Fraud Detection and Concept Drift Adaptation in Credit Card Transactions Using Incremental Gradient Boosting Trees, in: Wani M.A., Luo F., Li X., Dou D., Bonchi F. (Eds.), Proc. - IEEE Int. Conf. Mach. Learn. Appl., ICMLA, Institute of Electrical and Electronics Engineers Inc., 2020, pp. 545–550, journal Abbreviation: Proc. - IEEE Int. Conf. Mach. Learn. Appl., ICMLA. doi:10.1109/ICMLA51294.2020.00091.
URL https://www.scopus.com/pages/publications/85102496383?origin=resultslist

[3] Y. Sun, Y. Sun, H. Dai, Two-stage cost-sensitive learning for data streams with concept drift and class imbalance, IEEE Access 8 (2020) 191942–191955. doi:10.1109/ACCESS.2020.3031603.
URL https://www.scopus.com/pages/publications/85102806727?origin=resultslist

[4] L. Loezer, F. Enembreck, J. P. Barddal, A. de Souza Britto, Cost-sensitive learning for imbalanced data streams, in: Proceedings of the 35th Annual ACM Symposium on Applied Computing, SAC '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 498–504. doi:10.1145/3341105.3373949.
URL https://doi.org/10.1145/3341105.3373949

[5] G. J. Aguiar, B. Krawczyk, A. Cano, Cost-sensitive adaptive random forest for imbalanced data streams, IEEE Transactions on Knowledge and Data Engineering (2023).

[6] S. Wang, L. L. Minku, X. Yao, Resampling-based ensemble methods for

online class imbalance learning, IEEE Transactions on Knowledge and Data Engineering 27 (5) (2015) 1356–1368. doi:10.1109/TKDE.2014.2345380.

[7] T. Peng, Y.-J. Cheng, Hybrid Architecture Using Deep Auto-Encoder and Clustering Methods to Identify Diverse Transaction Anomalies, in: Meen T.-H. (Ed.), Proc. IEEE Int. Conf. Comput., Big-Data Eng., ICCBE, Institute of Electrical and Electronics Engineers Inc., 2025, pp. 584–589, journal Abbreviation: Proc. IEEE Int. Conf. Comput., Big-Data Eng., ICCBE. doi:10.1109/ICCBE65177.2025.11255690.
URL https://www.scopus.com/pages/publications/105031120508?origin=resultslist

[8] A. Bernardo, H. M. Gomes, J. Montiel, B. Pfahringer, A. Bifet, E. D. Valle, C-smote: Continuous synthetic minority oversampling for evolving data streams, in: 2020 IEEE International Conference on Big Data (Big Data), 2020, pp. 483–492. doi:10.1109/BigData50022.2020.9377768.

[9] A. Bernardo, E. Valle, SMOTE-OB: Combining SMOTE and Online Bagging for Continuous Rebalancing of Evolving Data Streams, in: Chen Y., Ludwig H., Tu Y., Fayyad U., Zhu X., Hu X.T., Byna S., Liu X., Zhang J., Pan S., Papalexakis V., Wang J., Cuzzocrea A., Ordonez C. (Eds.), Proc. - IEEE Int. Conf. Big Data, Big Data, Institute of Electrical and Electronics Engineers Inc., 2021, pp. 5033–5042, journal Abbreviation: Proc. - IEEE Int. Conf. Big Data, Big Data. doi:10.1109/BigData52589.2021.9671609.

URL https://www.scopus.com/pages/publications/85125321474?origin=resultslist

[10] C. W. Chiu, L. L. Minku, SMOClust: Synthetic minority oversampling based on stream clustering for evolving data streams, Machine Learning 113 (7) (2024) 4671–4721. doi:10.1007/s10994-023-06420-y.

[11] V. Suggula, R. R. Goli, Real-time fraud detection in banking transactions using kafka streaming and machine learning ensemble, in: 2025 1st International Conference on Advancement in Futuristic Technologies (ICAFT), 2025, pp. 1–7. doi:10.1109/ICAFT66710.2025.11453240.

[12] R. Attivilli, A. Arul Jothi, Serverless Stream-Based Processing for Real Time Credit Card Fraud Detection Using Machine Learning, in: Chakrabarti S., Paul R. (Eds.), IEEE World AI IoT Congr., AI-IoT, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 434–439, journal Abbreviation: IEEE World AI IoT Congr., AIIoT. doi:10.1109/AIIoT58121.2023.10174314.
URL https://www.scopus.com/pages/publications/85166669608?origin=resultslist

[13] G. Venkatasubbu, Streaming intelligence real time fraud detection in retail payments using machine learning, in: 2025 International Conference on Engineering and Emerging Technologies (ICEET), 2025, pp. 1–6. doi:10.1109/ICEET67911.2025.11424068.

[14] P. Tomar, S. Shrivastava, U. Thakar, Ensemble learning based credit card fraud detection system, in: 2021 5th Conference on Information

and Communication Technology (CICT), 2021, pp. 1–5. doi:10.1109/CICT53865.2020.9672426.

[15] D. Dhiman, A. Bisht, A. Kumari, D. Anandaram, S. Saxena, K. Joshi, Online Fraud Detection using Machine Learning, in: Int. Conf. Artif. Intell. Smart Commun., AISC, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 161–164, journal Abbreviation: Int. Conf. Artif. Intell. Smart Commun., AISC. doi:10.1109/AISC56616.2023.10085493.
URL https://www.scopus.com/pages/publications/85153506062?origin=resultslist

[16] K. Sudarshana, C. MylaraReddy, Z. Adhoni, Classification of Credit Card Frauds Using Autoencoded Features, in: Rao B.N.K., Balasubramanian R., Wang S.-J., Nayak R. (Eds.), Smart Innov. Syst. Technol., Vol. 315, Springer Science and Business Media Deutschland GmbH, 2023, pp. 9–17, journal Abbreviation: Smart Innov. Syst. Technol. doi:10.1007/978-981-19-4162-7_2.
URL https://www.scopus.com/pages/publications/85142669157?origin=resultslist

[17] H. Li, Credit Card Fraud Detection Based on Combination of Sparse Autoencoder and Support Vector Machine, in: ACM Int. Conf. Proc. Ser., Association for Computing Machinery, 2022, pp. 1252–1255, journal Abbreviation: ACM Int. Conf. Proc. Ser. doi:10.1145/3573428.3573650.
URL https://www.scopus.com/pages/publications/85150486090?origin=resultslist

[18] K. Al-Daoud, I. Abu-AlSondos, Robust AI for Financial Fraud Detection in the GCC: A Hybrid Framework for Imbalance, Drift, and Adversarial Threats, J. Theor. Appl. Electron. Commer. Res. 20 (2) (2025). doi:10.3390/jtaer20020121.
URL https://www.scopus.com/pages/publications/105009131920?origin=resultslist

[19] C.-T. Chen, C. Lee, S.-H. Huang, W.-C. Peng, Credit Card Fraud Detection via Intelligent Sampling and Self-supervised Learning, ACM Trans. Intell. Syst. Technolog. 15 (2) (2024) 1–29. doi:10.1145/3641283.
URL https://www.scopus.com/pages/publications/85189864744?origin=resultslist

[20] F. Moradi, M. Tarif, M. Homaei, Ensemble-based fraud detection: A robust approach evaluated on ieee-cis, in: 2025 15th International Conference on Computer and Knowledge Engineering (ICCKE), 2025, pp. 1–6. doi:10.1109/ICCKE68588.2025.11273852.

[21] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Comput. Surv. 46 (4) (Mar. 2014). doi:10.1145/2523813.
URL https://doi.org/10.1145/2523813

[22] J. Liu, X. Li, W. Zhong, Ambiguous decision trees for mining concept-drifting data streams, Pattern Recognition Letters 30 (15) (2009) 1347–1355. doi:10.1016/j.patrec.2009.07.017.

[23] Z. Cui, H. Tian, H. Shen, Effective Density-Based Concept Drift De-

tection for Evolving Data Streams, in: Park J.S., Takizawa H., Shen H., Park J.J. (Eds.), Lect. Notes Electr. Eng., Vol. 1112 LNEE, Springer Science and Business Media Deutschland GmbH, 2024, pp. 190–201, journal Abbreviation: Lect. Notes Electr. Eng. doi:10.1007/978-981-99-8211-0_18.
URL https://www.scopus.com/pages/publications/85180155927?origin=resultslist

[24] H. Al Lawati, A. Zainal, B. Al-Rimy, M. Al-Azawi, M. Kassim, S. Almalki, T. Alghamdi, An Integrated Preprocessing and Drift Detection Approach With Adaptive Windowing for Fraud Detection in Payment Systems, IEEE Access 13 (2025) 92036–92056. doi:10.1109/ACCESS.2025.3569609.
URL https://www.scopus.com/pages/publications/105005082418?origin=resultslist

[25] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, T. Abdessalem, Adaptive random forests for evolving data stream classification, Machine Learning 106 (9) (2017) 1469–1495.

[26] Visa Inc., VisaNet Fact Sheet, https://usa.visa.com/dam/VCOM/download/corporate/media/visanet-technology/aboutvisafactsheet.pdf (2023).

[27] M. Abbasi, S. Lopez Florez, A. Shahraki, A. Taherkordi, J. Prieto, J. Corchado, Class Imbalance in Network Traffic Classification: An

Adaptive Weight Ensemble-of-Ensemble Learning Method, IEEE Access 13 (2025) 26171–26192. doi:10.1109/ACCESS.2025.3538170.
URL https://www.scopus.com/pages/publications/85217560478?origin=resultslist

[28] S. Priya, A. Uthra, Adaptive Synthetic Oversampling Algorithm for Handling Class Imbalance in Multi-Class Data Stream Classification, J. Comput. Sci. 18 (7) (2022) 650–664. doi:10.3844/jcssp.2022.650.664.
URL https://www.scopus.com/pages/publications/85135517854?origin=resultslist

[29] Y. Sun, M. Li, L. Li, H. Shao, Y. Sun, Cost-Sensitive Classification for Evolving Data Streams with Concept Drift and Class Imbalance, Comput. Intell. Neurosci. 2021 (2021). doi:10.1155/2021/8813806.
URL https://www.scopus.com/pages/publications/85113855601?origin=resultslist

[30] Y. Chen, X. Yang, H.-L. Dai, Cost-sensitive continuous ensemble kernel learning for imbalanced data streams with concept drift, Knowl Based Syst 284 (2024). doi:10.1016/j.knosys.2023.111272.
URL https://www.scopus.com/pages/publications/85180410042?origin=resultslist

[31] P. Domingos, G. Hulten, Mining high-speed data streams, in: Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining, 2000, pp. 71–80.

[32] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, G. Bontempi, Credit card fraud detection: a realistic modeling and a novel learning strategy, IEEE transactions on neural networks and learning systems 29 (8) (2018) 3784–3797.

[33] E. Lopez-Rojas, A. Elmir, S. Axelsson, Paysim : A financial mobile money simulator for fraud detection, in: 28th European Modeling and Simulation Symposium, EMSS 2016, Dime University of Genoa, 2016, pp. 249–255.

[34] A. Howard, B. Bouchon-Meunier, I. CIS, inversion, J. Lei, Lynn@Vesta, Marcus2010, P. H. Abbass, Ieee-cis fraud detection, `https://kaggle.com/competitions/ieee-fraud-detection`, kaggle (2019).

[35] E. A. Lopez-Rojas, S. Axelsson, Banksim: A bank payments simulator for fraud detection research, in: 26th European Modeling and Simulation Symposium, EMSS 2014, Dime University of Genoa, 2014, pp. 144–152.

[36] A. Bifet, R. Gavald'a, Adaptive learning from evolving data streams, in: Advances in Intelligent Data Analysis VIII: 8th International Symposium on Intelligent Data Analysis, IDA 2009, Lyon, France, August 31-September 2, 2009. Proceedings 8, Springer, 2009, pp. 249–260. `doi:10.1007/978-3-642-03915-7_22`.

[37] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine Learning Research 7 (Jan) (2006) 1–30.